

Binary Tree Representations:

The following are the various methods of implementing binary trees in C.

1. Node representation of Binary Trees.
2. Dynamic Node representation of Binary Trees.
3. Implicit Array representation of Binary Trees.

1. Node representation of Binary Trees.

The tree nodes may be implemented as array elements or as allocations of a dynamic variable. Each node contains info, left, right and parent fields. The left, right and parent fields of a node point to the node's left child node, right child node and parent node respectively.

Using the array implementation, we may declare:

```
# define NUMNODES 500
```

```
Struct nodetype
```

```
{
```

```
    int info;
```

```
    int left;
```

```
    int right;
```

```
    int parent;
```

```
};
```

```
Struct nodetype node[NUMNODES];
```

Under this representation, the operations info(p), left(p), right(p) and parent(p) are implemented by references to node[p].info, node[p].left, node[p].right and node[p].parent respectively.

Each node in a tree is taken from the available pool when needed and returned to the available pool when no longer in use. This representation is also called linked array representation.

2. Dynamic Node Representation of a Binary tree:

In this a node may be defined as follows:

```
Struct node
{
int info;
Struct node *left;
Struct node *right;
Struct node *parent;
};
```

```
typedef struct node * NODEPTR;
```

The operations info(p), left(p), right(p) and parent(p) would be implemented by reference to $p \rightarrow \text{info}$, $p \rightarrow \text{left}$, $p \rightarrow \text{right}$ and $p \rightarrow \text{parent}$ respectively.

The nodes are created using dynamic memory allocation.

If a tree is always traversed in downward fashion (from the root to the leaves), the parent operation is never used, so parent field is unnecessary.

The following is the definition for the node in top-down traversing.

```
Struct node
{
int info;
Struct node *left;
Struct node *right;
};
typedef struct node *NODEPTR;
```

The following program is to insert an element into a binary search tree and to find duplicate numbers in an input file in which each number is on a separate input line.

```
Struct node
{
int info;
Struct node *left;
Struct node *right;
};
typedef struct node *NODEPTR;
```

```

main()
{
NODEPTR root, p, q;
Int number;
scanf("%d" &number);
root = maketree(number);
while(scanf("%d", &number)!=EOF)
{
p=q=root;
while(number!=p→info && q!=NULL)
{
p=q;
if(number < p→info)
q=p→left;
else
q=p→right;
}
if(number == p→info)
printf("%d is duplicate \n", number);
else if(number<p→info)
setleft(p, number);
else
setright(p, number);
}
}

```

The definition for the function maketree is given as follows:

```

NODEPTR maketree(int x);
{
NODEPTR p;
p=getnode();
p→info=x;
p→left = NULL;
p→right = NULL;
return(p);
}

```

The definition for the function setleft() is given as follows:

```

void setleft(NODEPTR p, int x)
{

```

```

if (p==NULL)
    printf("cannot Insert\n");
else if(p->left !=NULL)
    printf("Invalid Insertion\n");
else
    p->left = maketree(x);
}

```

The definition for the function setright() is given as follows:

```

void setright(NODEPTR p, int x)
{
    if (p==NULL)
        printf("cannot Insert\n");
    else if(p->right !=NULL)
        printf("Invalid Insertion\n");
    else
        p->right = maketree(x);
}

```

3. Implicit Array Representation of Binary Trees:

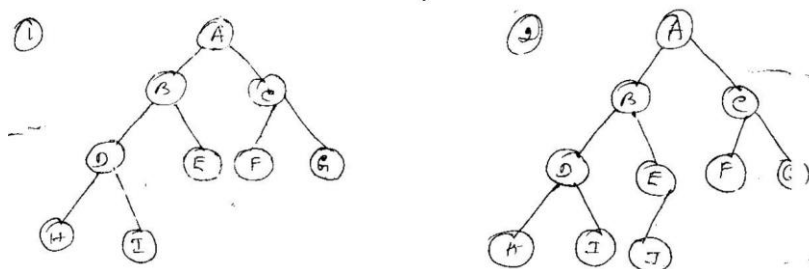
We can represent an almost complete binary tree without parent, left or right links. Instead, the nodes can be kept in an array of size specified.

In C, arrays start at position 0, therefore we number nodes from 0 to n-1.

So the left child of node numbered p is in position $2p+1$. The right child of node numbered p is $2p+2$.

The root of the tree is at position 0.

Example: Consider the following almost complete binary tree.



The implicit array representation of the above is given as follows:

1. →

0	1	2	3	4	5	6	7	8
A	B	C	D	E	F	G	H	I

2. →

0	1	2	3	4	5	6	7	8	9
A	B	C	D	E	F	G	H	I	J

The implicit array representation is also called sequential representation.

We can extend this implicit array representation of almost complete binary trees to an implicit array representation of binary trees generally.

The following figure shows 2 non almost complete binary trees and this equivalent sequential representation.

Example:

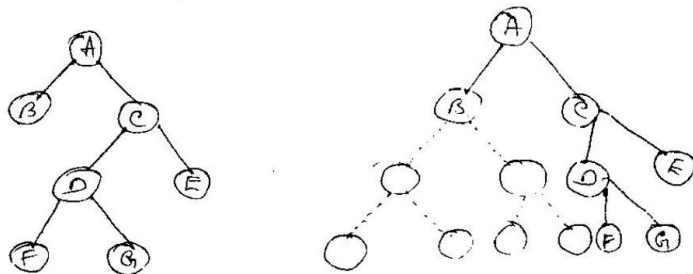
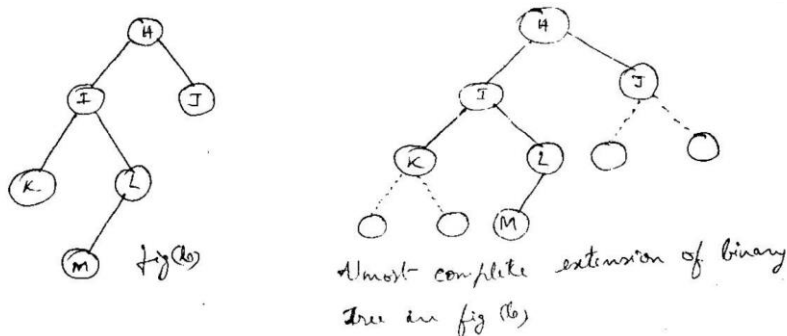


fig (a) Almost complete extension of binary tree in fig(a)
The sequential representation for the above is given by,

0	1	2	3	4	5	6	7	8	9	10	11	12
A	B	C				D	E				F	G



The sequential representation for the above is given by,

0	1	2	3	4	5	6	7	8	9	10	11	12
H	I	J	K	L								M